

Vehicle Routing Problems for Drone Delivery

Kevin Dorling, *Student Member, IEEE*, Jordan Heinrichs, Geoffrey G. Messier, *Member, IEEE*,
and Sebastian Magierowski, *Member, IEEE*

Abstract—Unmanned aerial vehicles, or drones, have the potential to significantly reduce the cost and time of making last-mile deliveries and responding to emergencies. Despite this potential, little work has gone into developing vehicle routing problems (VRPs) specifically for drone delivery scenarios. Existing VRPs are insufficient for planning drone deliveries: either multiple trips to the depot are not permitted, leading to solutions with excess drones, or the effect of battery and payload weight on energy consumption is not considered, leading to costly or infeasible routes. We propose two multitrip VRPs for drone delivery that address both issues. One minimizes costs subject to a delivery time limit, while the other minimizes the overall delivery time subject to a budget constraint. We mathematically derive and experimentally validate an energy consumption model for multirotor drones, demonstrating that energy consumption varies approximately linearly with payload and battery weight. We use this approximation to derive mixed integer linear programs for our VRPs. We propose a cost function that considers our energy consumption model and drone reuse, and apply it in a simulated annealing (SA) heuristic for finding suboptimal solutions to practical scenarios. To assist drone delivery practitioners with balancing cost and delivery time, the SA heuristic is used to show that the minimum cost has an inverse exponential relationship with the delivery time limit, and the minimum overall delivery time has an inverse exponential relationship with the budget. Numerical results confirm the importance of reusing drones and optimizing battery size in drone delivery VRPs.

Index Terms—Delivery, drone, heuristic, mixed integer program (MIP), simulated annealing (SA), traveling salesman problem (TSP), unmanned aerial vehicle (UAV), vehicle routing problem (VRP).

I. INTRODUCTION

UNMANNED aerial vehicles (UAVs), or drones, have the potential to significantly reduce the cost and time required to deliver packages. In general, drones are less expensive to maintain than traditional delivery vehicles such as trucks, and can lower labor costs by performing tasks autonomously. Delivering with drones may be faster than delivering with traditional delivery vehicles, as drones are not

limited by established infrastructure such as roads, and generally face less complex obstacle avoidance scenarios. The application of drones in emergency situations has the potential to save lives: drones can transport much-needed water, food, and medical supplies over hazardous terrain during a crisis, and can deploy wireless sensors to provide immediate updates on the event. This makes drones suitable not only for parcel delivery, but also for emergency response in the event of forest fires, oil spills, and earthquakes.

A number of large organizations have shown interest in drone delivery. In 2013, Amazon announced Prime Air [1], a service that utilizes multirotor drones to deliver packages from Amazon to customers. German logistics company Deutsche Post DHL also started its Parcelcopter project in 2013; the Parcelcopter [2] has transported medicine to the island of Juist in the North Sea. Google revealed Project Wing [3] in 2014 to produce drones that can deliver larger items than Prime Air and Parcelcopter. In 2014, the United Arab Emirates announced their plan to use drones to distribute official government documents such as permits and ID cards [4]. A startup called Matternet has partnered with Swiss Post to test a lightweight package delivery quadcopter [5].

Developments in numerous technologies have made it feasible for these organizations to perform drone deliveries. Carbon fiber manufacturing costs have decreased from 25 to 10 \$/kg over the last 20 years [6], [7], enabling the development of strong, lightweight airframes. Lithium polymer batteries, with their relatively high energy density [8], have improved the flight times of UAVs compared to alternative technologies such as nickel-cadmium and nickel-metal hydride. Drones typically use GPS to determine their location, and are able to take advantage of differential GPS [9] and localization techniques [10] to improve accuracy. Obstacles can be avoided through techniques such as light detection and ranging [11] and image processing [12]. Architectures and protocols have been developed that enable drones to form ad-hoc networks [13] and to wirelessly communicate with other entities [14].

Even though significant effort has gone into developing drone delivery technology, challenges unique to planning drone deliveries, such as the limited flight range and carrying capacity of drones, have been neglected in comparison. The flying sidekick traveling salesman problem (FSTSP) [15] provides one solution by allowing delivery trucks to also deploy drones, but relies on a combined fleet of trucks and drones to make deliveries. Green vehicle routing problems (GVRPs) [16] may account for battery-powered vehicles, but appear to be designed with trucks or other large capacity vehicles in mind.

Manuscript received January 18, 2016; revised April 6, 2016; accepted June 4, 2016. Date of publication July 14, 2016; date of current version December 14, 2016. This work was supported by the Natural Sciences and Engineering Research Council of Canada under Grant PGSD3-443041-2013. This paper was recommended by Associate Editor S. Nahavandi.

K. Dorling, J. Heinrichs, and G. G. Messier are with the Department of Electrical and Computer Engineering, University of Calgary, Calgary AB T2N 1N4, Canada (e-mail: kudorlin@ucalgary.ca; jheinric@ucalgary.ca; gmessier@ucalgary.ca).

S. Magierowski is with the Department of Electrical Engineering and Computer Science, York University, Toronto, ON M3J 1P3, Canada (e-mail: magiero@cse.yorku.ca).

Color versions of one or more of the figures in this paper are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/TSMC.2016.2582745

Drones have much smaller carrying capacities than trucks, and have limited flight times, meaning that in general they can only carry a small number of packages per route. The GVRP does not allow multiple trips to the depot, so as drone capacity and the maximum travel time decrease, it would likely compensate by unnecessarily purchasing more drones even though they could have been reused. We therefore propose solving drone delivery problems (DDPs) with a multitrip VRP (MTVRP) [17] that compensates for each drone's limited carrying capacity by reusing drones when possible.

The main contribution of this paper is the development of MTVRPs that consider battery and payload weight when calculating energy consumption. Other MTVRPs do not appear to account for battery and payload weight, limiting their applicability in drone delivery and emergency response scenarios. In Section VI-A, we experimentally show that increasing battery and payload weight can noticeably increase energy consumption, which in turn reduces flight time. Optimizing battery weight is important for drones: a drone's battery consumes a large portion of its carrying capacity, meaning that increasing its battery size to extend flight time significantly reduces the capacity available for packages. Approaches that consider the effect of payload weight on energy consumption have been shown in [18] to reduce costs in capacitated vehicle routing problems (CVRPs) compared to approaches that do not. Balancing payload weight, battery weight, and flight time are important considerations when attempting to minimize the cost or the delivery time for drone deliveries.

In addition to developing MTVRPs for drone delivery, we are the first to derive and experimentally validate a linear energy consumption model for multirotor drones. We demonstrate that energy consumption increases at an approximately linear rate with battery and payload weight. While others have developed models for large vehicles, we are not aware of any that exist for battery-powered drones. Our experiments not only validate our energy consumption model, but also demonstrate that drones consume approximately the same power regardless of being in hover or flying at a constant speed. We will show that this considerably simplifies our MTVRPs.

Our linear energy consumption model lets us derive mixed integer linear programs (MILPs) for solving the drone delivery MTVRPs. A linear energy consumption model allows for linear energy consumption constraints, which are required to derive an MILP compatible with fast commercial solvers such as CPLEX [19]. These problems optimize the number of drones, the routes they fly, as well as their battery weight, payload weight, and energy consumption. One VRP minimizes the cost of making the deliveries and is referred to as the minimum cost DDP (MC-DDP). Alternatively, in the case of an emergency response situation, a company may want to minimize overall delivery time, defined as the time it takes to complete all package deliveries; we call this the minimum time DDP (MT-DDP). We refer to the problems collectively as the DDPs.

As MILPs are NP-hard, large instances (in our case, scenarios with a large number of locations) may take a significant amount of time to solve. For such cases, we derive a simulated annealing (SA) heuristic for finding suboptimal solutions to the DDPs within a limited runtime. To help delivery network

planners understand how the DDPs behave under different conditions, we perform a sensitivity analysis to show how the optimal cost and overall delivery time found by the MC-DDP and MT-DDP, respectively, vary with the size of the area of interest, the number of locations considered, and the restrictions placed on the budget or overall delivery time. The SA heuristic is applied to demonstrate the importance of performing multiple trips and optimizing battery weight: VRPs that do not make such considerations may find costly or infeasible solutions to the DDPs.

While our DDPs allow for multiple deliveries per route, it is not clear if delivery companies such as Amazon and DHL are interested in delivering to more than one customer per route. The Amazon Prime Air website [1] and the DHL Parcelcopter press release [2] only show drones that can deliver single packages. Even if these companies only plan to deliver to one customer per route, our VRPs can be viewed as a generalization of this strategy. Constraints can be added to our problem to limit the number of customers per route. Furthermore, our method can be used to see if the savings from considering multiple deliveries per route make up for the costs associated with implementing the strategy.

The rest of this paper is organized as follows. In Section II, we review work relevant to the DDPs. Section III describes our energy consumption model for multirotor copters. In Section IV, we provide MILP implementations of the DDPs, while in Section V we propose an SA heuristic for finding suboptimal solutions to the problems within a constrained runtime. Section VI presents experimental results validating the fuel consumption model, evaluates the performance of our SA heuristic, and analyzes numerical results to gain insight on the DDPs. Our conclusions are provided in Section VII.

II. RELATED WORK

Drone routing papers, such as [20] and [21], typically focus on surveillance applications. The FSTSP [15] considers a delivery scenario based on combining drone delivery with a delivery truck. There do not appear to be papers that focus solely on using drones to make deliveries. This existing work on drone routing ignores factors important to drone delivery such as vehicle capacity, battery weight, changing payload weight, and reusing vehicles to reduce costs.

VRPs [22], [23] have been applied to solve delivery problems that, at first glance, appear similar to the DDPs proposed in this paper. A VRP attempts to find the optimal routes for one or more vehicles to deliver commodities to a set of locations. Each location may have a unique demand representing the number or size of commodities it requires, or a time window in which the vehicle should arrive. Vehicles typically leave from one or more depots, deliver their commodities, and then return to the depots. This section will explain why existing VRPs are not adequate for the DDP.

The GVRP, first described by Erdoğan and Miller-Hooks [16], allows vehicles to visit refuelling stations while making deliveries to extend their range. Schneider *et al.* [24] created a version for battery-powered delivery vehicles that considers time windows, capacity

constraints, and location demands. Hiermann *et al.* [25] built on Schneider *et al.*'s [24] work by allowing for a mixed fleet of various types of vehicles, perhaps with different capacities, battery sizes, and costs. They assume that charging infrastructure is in place, so vehicles can regain energy to extend their travel distance. While it could increase drone range, it is debatable whether delivery companies will deploy charging infrastructure for their drones, and unlikely that such infrastructure would be available in emergency response situations. In GVRPs, vehicles can visit the same charging station multiple times to restore energy, but appear to be able to visit the depot only once. Drones tend to have limited carrying capacities, so if the depot can only be visited once, a GVRP will likely require a large number of drones to satisfy every customer's demand. In Section VI-D, we show that reusing drones by allowing them to make multiple trips can significantly reduce costs.

A VRP that reuses vehicles is known as a multi-trip vehicle routing problem (MTVRP), first proposed by Fleischmann [26]. While a number of approaches are available for solving the MTVRP, such as a large neighborhood search algorithm [27], a hybrid genetic algorithm with a local search operator [28], a variable neighborhood search algorithm [29], and a branch-and-price algorithm [30], none appear to consider energy consumption as a function of vehicle weight. Routes found by the above approaches may be infeasible or unnecessarily costly in a drone delivery scenario, with batteries that are larger than necessary or too heavy to carry. In Section VI-E, we demonstrate that optimizing battery weight can reduce costs. Battery weight cannot be optimized without modeling energy consumption as a function of vehicle weight: the model is used to find the battery energy, and therefore weight, required to complete each route. The DDPs proposed in this paper apply our linear energy consumption model to optimize battery weight and payload weight, ensuring that the routes found are low-cost and feasible.

Our DDPs consider payload weight similar to the energy minimizing vehicle routing problem introduced by Kara *et al.* [31]. Kara *et al.* [31] utilized a VRP that factors in the effect of a vehicle's payload on total costs, but is based solely on Newtonian physics, and is not verified against an actual vehicle. Unlike [31], the work by Xiao *et al.* [18] provides a linear fuel consumption model for trucks based on actual fuel consumption measurements from [32], along with an SA algorithm for minimizing the cost of routes. The load dependant VRP [33] provides a local search heuristic for doing the same in pickup and delivery scenarios. While these VRPs consider the effect of payload weight on energy consumption, they appear to have been designed with vehicles that have relatively large capacities in mind, as they only let vehicles depart from the depot once. Drones have a limited carrying capacity, so if they can only leave the depot once, a large number will be required to satisfy demand in scenarios with many customers. To reduce the number and therefore cost of drones, we reuse them after they return to the depot.

The DDPs proposed in this paper combine MTVRPs with VRPs that model energy consumption as a function of vehicle weight to gain the strengths of both approaches. As mentioned

earlier, MTVRPs reduce drone costs by reusing drones after they return to the depot, but can propose routes that are costly or infeasible. VRPs that model energy consumption as a function of vehicle weight, on the other hand, ensure that individual routes are feasible and low-cost, but do not reuse vehicles to lower costs. The DDPs reduce drone costs by reusing vehicles *and* ensure low-cost, feasible routes by modeling energy consumption as a function of battery and payload weight.

III. MULTIROTOR HELICOPTER ENERGY CONSUMPTION

The flight time of a drone is limited by its weight and the energy stored in its battery. An energy consumption model helps balance the two by providing the energy consumed by the drone as a function of its weight. When optimizing deliveries, such a model can be used to compare the energy consumed by alternative routes.

While energy consumption equations are available for single rotor helicopters [34], to the best of our knowledge no such equations are available for multirotor helicopters, the type of drone that most delivery companies appear to favor. In this section, we derive an equation for the power consumed by a multirotor helicopter in hover as a function of its weight. We show that the power it consumes is approximately linearly proportional to the weight of its battery and payload under practical assumptions.

We derive the power consumed by a multirotor drone in hover, but not during flight, takeoff, or landing. In flight, the power consumed by the helicopter is often reduced due to translational lift [34], a phenomena where air flowing horizontally along the rotor generates additional lift. The average power during hover is consequently an upper bound on the average power during flight. We assume that the power consumed during takeoff and landing is, on average, approximately equivalent to the power consumed during hover. We confirm in Section VI-A that these assumptions are realistic with a 3D Robotics ArduCopter Hexa-B hexacopter.

From [34, eq. (2.16)], we can calculate the power P^* in Watts of a single rotor helicopter in hover, with the thrust T in Newtons, fluid density of air ρ in kg/m^3 , and the area ζ of the spinning blade disc in m^2 using

$$P^* = \frac{T^{3/2}}{\sqrt{2\rho\zeta}} \quad (1)$$

where the thrust $T = (W + m)g$, given the frame weight W in kg, the battery and payload weight m in kg, and gravity g in N.

We can use (1) to derive an equation for the power consumed by an n -rotor copter if we assume that each rotor shares the total mass $W + m$ of the copter equally. Each rotor carries a weight of $m' = m/n$ for batteries and payload, and a frame weight of $W' = W/n$, meaning that the power consumed by a single rotor is

$$P' = (W' + m')^{3/2} \sqrt{\frac{g^3}{2\rho\zeta}}.$$

So the power consumed by all n rotors is

$$P = nP' = (W + m)^{3/2} \sqrt{\frac{g^3}{2\rho\zeta n}}. \quad (2)$$

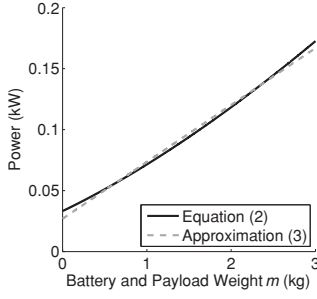


Fig. 1. Linear approximation (3) fitted to the power consumption model (2), assuming $n = 6$ rotors, a fluid density of $\rho = 1.204 \text{ kg/m}^3$, a rotor disc area of $\varsigma = 0.2 \text{ m}^2$, and a frame weight $W = 1.5 \text{ kg}$.

Note that the inverse relationship between P and n seen in (2) is a result of n increasing the effective disc area.

The linear approximation of the n -rotor copter power consumption equation (2) can be expressed as

$$p(m) = \alpha m + \beta \quad (3)$$

where α represents the power consumed per kilogram of battery and payload weight m , and β is the power required to keep the hexacopter frame in the air. In Section IV, we use (3) to develop MILPs for optimizing the routes flown by a set of drones to minimize the time and cost required to deliver packages to a set of locations. We will see that the linear approximation is necessary for the constraints in the MILPs to be linear. Linear constraints ensure compatibility with the majority of mixed integer program (MIP) solvers available.

Fig. 1 shows that (3) closely fits (2), assuming a Hexa-B hexacopter such that $n = 6$, $\rho = 1.204 \text{ kg/m}^3$, $\varsigma = 0.2 \text{ m}^2$, and $W = 1.5 \text{ kg}$. Applying a linear regression to (2) for $m = 0\text{--}3 \text{ kg}$ in increments of 0.001 kg results in $\alpha = 46.7 \text{ W/kg}$ and $\beta = 26.9 \text{ W}$. Our linear approximation (3) closely fits the exact equation (2), with a mean percent error of 3.1% and the largest difference being 6.3 W . In Section VI-A, we validate this approximation experimentally with a Hexa-B hexacopter.

The linear approximation will likely hold for other multirotor helicopters. The parameters n , ς , ρ , and W are presumably similar between helicopters and are all under a square root in (2), meaning slight adjustments to them will have a limited impact on P . A larger variation in m , however, can significantly reduce the accuracy of the approximation: if m varies between 0 and 10 kg , the mean percent error becomes 12.8% , and the largest difference between (2) and (3) is about 51 W .

IV. DRONE DELIVERY PROBLEMS

In this section, we apply (3) to develop MILPs for optimizing the number of drones in a fleet, as well as the routes they fly, in order to solve the DDPs. When the objective is to minimize costs, we refer to the problem as the MC-DDP. A company that specializes in delivering parcels may prioritize cost minimization over time. In an emergency response situation, however, a company could instead prioritize time, and would consequently solve the MT-DDP. In both DDPs, constraints on the budget B and delivery time limit T may be imposed.

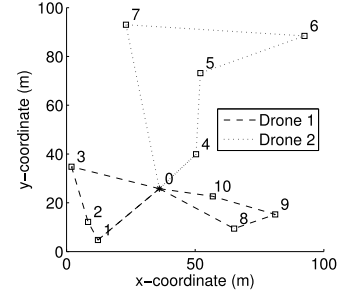


Fig. 2. Example showing two drones flying three different routes.

We define a route as a path that starts and ends at the depot. The example given in Fig. 2 depicts two drones flying three routes. Whilst drone 2 flies a single route, visiting locations 4–7, drone 1 flies two routes. During its first route, drone 1 visits locations 1–3, then returns to the depot to swap batteries and pick up more packages. After the swap, it flies its second route, visiting locations 8–10. We assume that only one depot exists in the area. The depot acts like a charging station: each vehicle may return to the depot multiple times to collect more packages and replace its batteries. The DDPs optimize the routes flown by a set of drones in order to deliver packages to a set of locations $\mathcal{N}$. Location 0 is the depot. Each location $i \in \mathcal{N}_0$, where the set $\mathcal{N}_0 = \mathcal{N} \setminus \{0\}$, has a demand D_i which represents the weight of the package in kg that will be delivered to location i . Every location, except for the depot, is visited only once by a drone, and $\tau \text{ s}$ is spent at each location to descend, deliver the package, and ascend.

The rest of this section is dedicated to explaining our assumptions and deriving the MILP forms of the MT-DDP and MC-DDP. We discuss our assumptions in Section IV-A. In Section IV-B, we list the constraints and decision variables used in the DDPs. Section IV-C provides each DDP's objective function and MILP formulation.

A. Assumptions

This section explains the assumptions we make regarding drone flight speed, battery charging, and the number of depots. We assume that drones fly between locations at a constant speed v in m/s. In Section VI-A, we demonstrate that while the energy consumed by a hexacopter during flight is approximately the same as the energy consumed during hover, the energy consumed during flight can actually be slightly lower than the energy consumed in hover, likely due to translational lift. We therefore assume that the operator would fly drones as fast as they reasonably can, at a constant speed of $v \text{ m/s}$ between locations.

Note that one weakness of this assumption is that it ignores the impact of weather. Flying with the wind, for example, could reduce energy consumption. Cold temperatures may adversely affect battery performance until the batteries warm up. For the sake of generality, we ignore the effects of weather. Understanding how hexacopter speed and energy consumption are affected by the weather would likely require additional measurements in different weather conditions.

We assume that the demand at each location can be fully satisfied by a single drone. If the demand at a location is higher than the drone carrying capacity, the DDPs do not make multiple drones deliver optimally to that location. To overcome this issue, multiple locations can be placed on top of one another. Their combined demand can then be higher than the drone's carrying capacity. While this does not necessarily find the optimal split, it at least lets drones complete the delivery.

We do not consider recharging batteries after they have been swapped out of a drone when calculating energy costs. Instead we assume that the operator will purchase enough fully charged batteries to satisfy drone energy requirements before deliveries commence. For a delivery company, managing the charging of various battery sizes between trips could be difficult and costly compared to allowing the batteries to charge overnight. In addition, batteries have a limited number of charge cycles, so charging them between trips would likely not provide long-term savings, as they would have to be replaced more frequently. Disaster management scenarios require low response times, limiting charging time between trips.

We do not consider multiple depots. While doing so could extend the range of the drones and potentially save money, we believe considering multiple depots for drone delivery would be handled better in a separate paper that discusses optimizing the number of depots, their distribution over an area, as well as the allocation of batteries and packages among them. The survey paper [35] discusses VRPs for multiple depots.

B. Decision Variables and Constraints

This section discusses the decision variables, constants, and constraints used in the DDPs. The DDPs have identical constraints, which can be organized into categories related to limiting each drone's route, reusability, timing, energy consumption, capacity, as well as the total cost of making deliveries. Decision variables and constants are described below their corresponding constraints.

We ensure that every route is valid through

$$\sum_{\substack{j \in \mathcal{N} \\ i \neq j}} x_{ij} = 1 \quad \forall i \in \mathcal{N}_0 \quad (4a)$$

$$\sum_{\substack{j \in \mathcal{N} \\ i \neq j}} x_{ij} - \sum_{\substack{j \in \mathcal{N} \\ i \neq j}} x_{ji} = 0 \quad \forall i \in \mathcal{N}. \quad (4b)$$

To create a routing map for the drones, we use the edge variables x_{ij} , where $x_{ij} = 1$ if the drone moves from location i to j , and $x_{ij} = 0$ otherwise. Constraint (4a) guarantees that every location, except for the depot, is visited exactly once by a drone, while (4b) ensures that a drone arriving at location i also departs from location i .

The reusability constraints

$$\sum_{j \in \mathcal{N}_0} \sigma_{ij} \leq x_{i0} \quad \forall i \in \mathcal{N}_0 \quad (5a)$$

$$\sum_{j \in \mathcal{N}_0} \sigma_{ji} \leq x_{0i} \quad \forall i \in \mathcal{N}_0 \quad (5b)$$

$$\sum_{i \in \mathcal{N}_0} x_{0i} - \sum_{\substack{(i,j) \in \mathcal{N}_0 \times \mathcal{N}_0 \\ i \neq j}} \sigma_{ij} \leq M \quad (5c)$$

determine whether or not a drone can be reused after returning to the depot. The reuse decision variable is σ_{ij} , where $\sigma_{ij} = 1$ if the drone leaves location i for the depot, gains a fresh battery and set of packages, then flies to location j to begin a new route; otherwise $\sigma_{ij} = 0$. Constraint (5a) implies that if a drone returns to the depot from location i , it is available for use again to fly to another location. Constraint (5b) ensures that if a reused drone leaves from the depot to location i , it arrived previously from another location. The number of drones that can be purchased, and can therefore fly simultaneously, is limited to M by (5c).

We apply the demand constraints

$$\sum_{\substack{j \in \mathcal{N} \\ i \neq j}} y_{ji} - \sum_{\substack{j \in \mathcal{N} \\ i \neq j}} y_{ij} = D_i \quad \forall i \in \mathcal{N}_0 \quad (6a)$$

$$y_{ij} \leq Kx_{ij} \quad \forall (i,j) \in \mathcal{N} \times \mathcal{N}, \quad i \neq j \quad (6b)$$

to ensure that each location receives what it demands. The payload weight between locations i and j is represented by the decision variable y_{ij} in kg. The constant K is a large value representing an upper bound for constraints. Constraint (6a) makes sure that the payload weight when leaving location i is D_i kg less than upon arrival. Constraint (6b) sets the payload weight of each edge without a vehicle to 0 kg.

We calculate and enforce timing through

$$\begin{aligned} t_i - t_j + \tau + d_{ij}/v & \leq K(1 - x_{ij}) & \forall (i,j) \in \mathcal{N} \times \mathcal{N}_0 \\ & i \neq j \end{aligned} \quad (7a)$$

$$\begin{aligned} t_i - a_i + \tau + d_{i0}/v & \leq K(1 - x_{i0}) & \forall i \in \mathcal{N}_0 \end{aligned} \quad (7b)$$

$$\begin{aligned} a_i - t_j + \tau + d_{0j}/v & \leq K(1 - \sigma_{ij}) & \forall (i,j) \in \mathcal{N}_0 \times \mathcal{N}_0 \\ & i \neq j \end{aligned} \quad (7c)$$

$$t_i \leq l \quad \forall i \in \mathcal{N}_0 \quad (7d)$$

$$l \leq T. \quad (7e)$$

A location $i \in \mathcal{N}_0$ is visited by a drone at time t_i in seconds. The time in seconds that a drone returns to the depot directly after leaving location i is a_i . Note that $a_i = 0$ if $x_{i0} = 0$, and $a_i > 0$ otherwise. The overall delivery time l is the time in seconds when all drones have completed their deliveries. Constant values related to timing include the speed v in m/s of the drones in the air, the distance d_{ij} in m between locations i and j , as well as the time τ in seconds spent at each location descending, delivering a package, and ascending. Drones must complete their deliveries by the delivery time limit T in seconds. Constraint (7a) keeps track of the time t_i that each location i is visited by a drone. Similarly, (7b) keeps track of the time a_i that a drone arrives at the depot from location i . Constraint (7c) ensures that times are correct for drones that are reused after returning to the depot. The overall delivery time l is set by (7d), and the delivery time limit T is guaranteed by (7e).

Carrying capacity is restricted through

$$q_{ij} + y_{ij} \leq Qx_{ij} \quad \forall (i, j) \in \mathcal{N} \times \mathcal{N} \quad (8a)$$

$$i \neq j$$

$$z_i/\xi - \zeta_i \leq K(1 - x_{i0}) \quad \forall i \in \mathcal{N}_0 \quad (8b)$$

$$\zeta_i - \zeta_j \leq K(1 - x_{ji}) \quad \forall (i, j) \in \mathcal{N}_0 \times \mathcal{N}_0 \quad (8c)$$

$$i \neq j$$

$$q_{ij} \geq \zeta_j - K(1 - x_{ij}) \quad \forall (i, j) \in \mathcal{N} \times \mathcal{N}_0 \quad (8d)$$

$$i \neq j$$

$$q_{i0} \geq \zeta_i - K(1 - x_{i0}) \quad \forall i \in \mathcal{N}_0. \quad (8e)$$

The battery weight between locations i and j is represented by the decision variable q_{ij} in kg. To assist with optimizing the battery weight, the decision variable ζ_i tracks the battery weight in kg at location i . The energy consumed from a drone's battery by the time it arrives at the depot directly after leaving location i is z_i kJ. Note that $z_i = 0$ if $x_{i0} = 0$, while $z_i > 0$ otherwise. The constant ξ is the energy density of the battery in kJ/kg. The capacity of the drone between locations i and j is restricted to Q kg by (8a). The weight ζ_i of the battery at each location i is set by (8b) and (8c): constraint (8b) finds ζ_i for the locations visited just before the depot, while (8c) sets $\zeta_i = \zeta_j$ if the drone flies directly from location i to location j . The weight q_{ij} of the battery between locations i and j is required by (8a) and is found through constraints (8d) and (8e). Constraint (8d) sets $q_{ij} \geq \zeta_j$ if the drone flies between locations i and j , while (8e) sets $q_{i0} \geq \zeta_i$ if the drone flies from location i to the depot.

Energy restrictions are enforced by

$$f_i - f_j + p(m_{ij})(d_{ij}/v + \tau) \quad \forall (i, j) \in \mathcal{N} \times \mathcal{N}_0 \quad (9a)$$

$$\leq K(1 - x_{ij}) \quad i \neq j$$

$$f_i - z_i + p(m_{i0})(d_{i0}/v + \tau) \quad \forall i \in \mathcal{N}_0 \quad (9b)$$

$$\leq K(1 - x_{i0})$$

$$z_i \leq Kx_{i0} \quad \forall i \in \mathcal{N}_0. \quad (9c)$$

The decision variable f_i represents the energy in kJ consumed from a drone's current battery upon reaching location $i \in \mathcal{N}_0$. The power $p(m)$ in kW consumed by a drone with a battery and payload weight of m kg is the linear approximation (3). The weight in kg of the drone's battery and payload between locations i and j is $m_{ij} = q_{ij} + y_{ij}$. Constraint (9a) forces f_i to equal the total energy consumed along the route up to location i . Constraint (9b) makes z_i equal to the energy consumed flying the entire route that ends at location i . To ensure that $z_i = 0$ if the drone does not fly from location i to the depot, and that $z_i > 0$ otherwise, we include (9c). Note that constraints (9a) and (9b) are linear because the power $p(m_{ij})$ is the linear approximation (3).

Costs are kept in line with the budget B through

$$c = F \sum_{i \in \mathcal{N}_0} x_{i0} - F \sum_{\substack{(i,j) \in \mathcal{N}_0 \times \mathcal{N}_0 \\ i \neq j}} \sigma_{ij} + \epsilon \sum_{i \in \mathcal{N}_0} z_i \quad (10a)$$

$$c \leq B. \quad (10b)$$

The cost of a drone is F financial units, while the budget is limited to B financial units. The constant ϵ is the cost in

financial units of a kJ of energy. Constraint (10a) calculates the total cost c of performing the deliveries. The leftmost term of (10a) represents the cost of drones assuming that each route requires a new drone, while the second term represents the savings provided by reusing drones; together they equal the total cost of drones. The rightmost term is the cost of energy. The total cost is restricted to the budget by (10b).

The constraints in this section assume each drone's battery is sized to provide exactly enough energy for the upcoming route. The constraints can, however, be adjusted to find the optimum combination of discrete battery sizes. Assume a set of battery types $\mathcal{B}$ exists, where each battery type $j \in \mathcal{B}$ has energy E_j in kJ, a cost C_j in financial units, a weight w_j in kg, and a decision variable b_{ji} that is 1 if battery type j is in the drone at location i with $z_i \geq 0$, and 0 otherwise. In (8b), the continuous battery weight z_i/ξ at location i can be replaced with the weight of the chosen batteries $\sum_{j \in \mathcal{B}} w_j b_{ji}$. In (10a), the continuous total cost $\epsilon \sum_{i \in \mathcal{N}_0} z_i$ of batteries can be replaced with the total cost of the chosen batteries $\sum_{i \in \mathcal{N}_0} \sum_{j \in \mathcal{B}} C_j b_{ji}$. To ensure that the chosen batteries' energy is adequate, constraint $\sum_{j \in \mathcal{B}} E_j b_{ji} \geq z_i, \forall i \in \mathcal{N}_0$ can be added.

C. Mixed Integer Linear Program Formulation

This section provides MILP formulations of the DDPs. As both problems have identical decision variables and constraints, only their objective functions differ. By representing the problems in MILP form, we show that while the problems are nonconvex, they are compatible with commercial MIP solvers such as CPLEX [19].

The MT-DDP can be expressed as

$$\begin{aligned} \min \quad & l \\ \text{s.t.} \quad & (4a) \cdots (10b). \end{aligned} \quad (11)$$

The objective function (11) minimizes the overall delivery time l . According to (7d), the overall delivery time is equal to the time of the last package delivery.

The MC-DDP can be expressed as

$$\begin{aligned} \min \quad & c \\ \text{s.t.} \quad & (4a) \cdots (10b). \end{aligned} \quad (12)$$

The objective function (12) minimizes the total cost c , and therefore, according to (10a), the cost of drones and energy.

Both DDPs are nonconvex as they contain binary decision variables. However, the objective functions (11), (12) and the constraints (4a)–(10b), are all linear functions of the decision variables. This means the DDPs are MILPs compatible with most MIP solvers.

While MIP solvers find optimal solutions, the runtime can be large, even for problems with a small number of locations. In Section V, we propose an SA approach similar to the one in [18] for finding suboptimal solutions to the problem under limited runtimes.

V. SIMULATED ANNEALING IMPLEMENTATION

Even for scenarios with a small number of locations, the time required to solve the MILP implementation of the DDPs

Algorithm 1 COST(s, Φ) Method**Input:**

s - The solution vector
 Φ - A boolean variable that is 1 if the goal is to minimize cost, or 0 to minimize overall delivery time

Output:

c - The total cost of s
 l - The time taken by s to complete deliveries

```
// Find the cost of energy  $\lambda$ , the cost  $\gamma$  of drones, the overall
// delivery time  $l$ , and the total cost  $c$ .
 $\lambda \leftarrow \text{ENERGYCOST}(s)$ 
 $(\gamma, l) \leftarrow \text{DRONECOSTANDDELIVERYTIME}(s, \lambda, \Phi)$ 
 $c \leftarrow \lambda + \gamma$ 
// Enforce budget and delivery time limit by increasing cost
// and overall delivery time if either constraint is violated.
if  $c > B$  then
     $c \leftarrow c + K(c - B)$ 
     $l \leftarrow l + K(c - B)$ 
else if  $l > T$  then
     $c \leftarrow c + K(l - T)$ 
     $l \leftarrow l + K(l - T)$ 
return  $(c, l)$ 
```

to optimality may be prohibitively long. Under time constraints, a suboptimal solution may be preferable. In this section, we create a function for determining the cost and overall delivery time of a DDP solution. This cost function takes into account the cost and weight of batteries, the payload weight of each drone, and the fact that each drone can perform multiple trips. To solve practical problems with large numbers of locations, we apply it to an SA [36] heuristic similar to the algorithms in [18] and [37].

We take a string-based approach, meaning that a DDP solution is represented as a 1-D vector of whole numbers $s = [0 \ r_1 \ 0 \ r_2 \ 0 \ \dots \ r_R \ 0]$, where, as described in [37], the number 0 represents the depot, r_n represents a route flown by a drone, and R is the number of routes. The variable r_n is a vector whose k th element $r_{nk} \in \mathcal{N}$, where $\mathcal{N}$ is the set of locations. The routes in Fig. 2 could, for example, be represented by the string $s = [0 \ 1 \ 2 \ 3 \ 0 \ 0 \ 8 \ 9 \ 10 \ 0 \ 4 \ 5 \ 6 \ 7 \ 0]$. When two neighboring zeros occur, there is an empty route; a total of three routes exist in the above s , even though five could have been flown. Reference [37, Sec. 2.3] discusses the string-based model and its advantages in greater detail.

The rest of this section is dedicated to explaining our SA implementation. To decide whether one solution is an improvement over another, we compare their costs. The method we use to determine the cost of a solution is presented in Section V-A. Finally, the SA algorithm is discussed in Section V-B.

A. DDP Cost Function

The cost of a set of routes depends on the type of DDP being solved. In the MT-DDP the best solution has the lowest overall delivery time, while in the MC-DDP the best solution has the lowest cost. In this section, we create a function for calculating the cost and overall delivery time of a DDP solution.

Algorithm 2 ENERGYCOST(s) Method**Input:**

s - The solution vector

Output:

λ - The cost of energy for solution s

```
// Initialize the time  $t$ , sum-of-products  $\omega$  of time and weight,
// payload weight  $y$ , and energy cost  $\lambda$ .
 $t \leftarrow 0$ 
 $\omega \leftarrow 0$ 
 $y \leftarrow 0$ 
 $\lambda \leftarrow 0$ 
// Iterate through the locations in  $s$  from back to front.
for  $k \in \{\text{LENGTH}(s)-2, \dots, 0\}$  do
    // Set the next location  $i$  and current location  $j$ .
     $i \leftarrow s_{k+1}$ 
     $j \leftarrow s_k$ 
    if  $i \neq 0 \vee j \neq 0$  then
        // Record the travel time  $\phi_{ij}$  between  $i$  and  $j$ .
         $\phi_{ij} \leftarrow \tau + d_{ij}/v$ 
        // Update the time, the sum-of-products of time and
        // weight, and the payload weight respectively.
         $t \leftarrow t + \phi_{ij}$ 
         $\omega \leftarrow \omega + y\phi_{ij}$ 
         $y \leftarrow y + D_j$ 
        // Update costs when a vehicle arrives at a depot.
        if  $(0 == j) \wedge (i \neq 0)$  then
             $E \leftarrow \text{BATTERYENERGY}(t, \omega)$ 
            // Add the cost of energy. Note that  $\epsilon$  is the cost
            // of a kJ. Greatly increase the cost if the positive
            // energy constraint is violated.
            if  $E > 0$  then
                 $\lambda \leftarrow \lambda + E\epsilon$ 
            else
                 $\lambda \leftarrow \lambda + -K(E\epsilon)$ 
            // Enforce the capacity constraint by greatly
            // increasing cost if the drone carries too much.
            if  $y + E/\xi > Q$  then
                 $\lambda \leftarrow \lambda + K(y + E/\xi - Q)$ 
        // Reset weight and time values for the next route.
         $t \leftarrow 0$ 
         $\omega \leftarrow 0$ 
         $y \leftarrow 0$ 
return  $\lambda$ 
```

We assume that violating the capacity constraint, budget constraint, or delivery time limit is costly. As in [18], we reduce the likelihood of accepting a solution that violates a constraint by penalizing it; we do so by greatly increasing the cost and overall delivery time. The constant K can be adjusted to reflect the actual costs of violating a constraint.

Algorithm 1 calculates the overall cost and delivery time of a solution s to the DDP. It takes the solution vector s and a boolean value Φ that is 1 for the MC-DDP and 0 for the MT-DDP as inputs. The algorithm is divided into two phases: the first phase calculates the cost of energy with Algorithm 2,

Algorithm 3 ROUTEDELIVERYANDARRIVALTIMES(s) Method**Input:**

s - The solution vector

Output:

u - A vector containing pairs of the delivery and arrival times for each route

// Initialize the delivery time h and arrival time a to 0.

h $\leftarrow$ 0

a $\leftarrow$ 0

// Initialize the route timing vector u to $\emptyset$.

u $\leftarrow \emptyset$

// Iterate through the locations in s from front to back.

for *k* $\in \{1 \dots \text{LENGTH}(s) - 1\}$ **do**

// Set the current location i and previous location j.

i $\leftarrow s_k$

j $\leftarrow s_{k-1}$

if *i* $\neq 0 \vee j \neq 0$ **then**

// Update the time variables.

h $\leftarrow a$

$\phi_{ij} \leftarrow \tau + d_{ij}/v$

a $\leftarrow a + \phi_{ij}$

// Add the delivery and arrival time to u.

if *i* $== 0 \wedge j \neq 0$ **then**

u $\leftarrow u \cup (h, a)$

// Reset the delivery time value.

a $\leftarrow 0$

return u

while the second finds the cost of drones and the overall delivery time with Algorithm 5. Later in this section we show that Algorithm 2 has a $\mathcal{O}(|\mathcal{N}_0|)$ bound, and Algorithm 5 has a $\mathcal{O}(|\mathcal{N}_0| \log^2 M)$ bound, where $|\mathcal{N}_0|$ is the number of locations in the area of interest, and M is the maximum number of drones that can be purchased. This means that Algorithm 1 has a $\mathcal{O}(|\mathcal{N}_0| \log^2 M)$ bound as well. The rest of this section discusses Algorithms 2–5.

Algorithm 2 finds the cost of energy by iterating over the solution vector *s*. It iterates from back to front, like the cost function in [18, Fig. A2]; if it went from front to back, it would have to calculate the initial value of payload weight *y* before iterating over a route, instead of setting *y* = 0. Algorithm 2 calls BATTERYENERGY(*t*, *ω*) to find the energy

$$\begin{aligned}
 E &= \sum_{k=a}^{b-1} p(y_{k(k+1)} + E/\xi) \phi_{k(k+1)} \\
 &= \frac{\alpha \sum_{k=a}^{b-1} y_{k(k+1)} \phi_{k(k+1)} + \beta \sum_{k=a}^{b-1} \phi_{k(k+1)}}{1 - (\alpha/\xi) \sum_{k=a}^{b-1} \phi_{k(k+1)}} \\
 &= \frac{\alpha \omega + \beta t}{1 - (\alpha/\xi) t}
 \end{aligned} \tag{13}$$

in kJ required by a drone to complete a route containing locations *a* to *b*. Note that *t* is the time taken to complete a route, *ω* is the sum-of-products of the travel time and payload weight between locations, *y_{ij}* is the payload weight in kg between locations *i* and *j*, *p*(*m*) is (3), *α* is the power in

Algorithm 4 LISTSCHEDULE(*u*, *n*) Method**Input:**

u - A vector containing pairs of delivery and arrival times for each route

n - The number of drones that can be assigned routes

Output:

l - The overall delivery time when the routes are assigned to *n* drones

// Apply the list scheduling algorithm to assign routes to drones. Initialize the drone timing vector κ.

κ $\leftarrow \emptyset$

for *i* $\in \{0 \dots n - 1\}$ **do**

κ $\leftarrow \kappa \cup (0, 0)$

// Go through every route in the route timing vector u.

for *i* $\in \{0 \dots \text{LENGTH}(u) - 1\}$ **do**

// Get the delivery and arrival times of the drone with the smallest arrival time. Then assign route i to that drone.

$(h', a') \leftarrow \text{POPMINIMUMARRIVALTIMEELEMENT}(\kappa)$

$(h, a) \leftarrow u_i$

$\text{INSERTELEMENT}(\kappa, (a' + h, a' + a))$

// Return the maximum delivery time in κ.

return MAXIMUMDELIVERYTIME(*κ*)

kW consumed per kg of battery and payload weight, *β* is the power in kW required to keep the drone in hover assuming no battery or payload weight, *ξ* is the battery energy density in kJ/kg, and *φ_{ij}* is the travel time in seconds between locations *i* and *j*. As Algorithm 2 iterates through *s* once, its bound is $\mathcal{O}(|\mathcal{N}_0|)$.

Algorithm 2 assumes that battery weight and cost can be sized to provide the exact amount of energy required. If it is preferable to combine discrete battery sizes instead, it can be shown that finding the optimal combination of batteries that minimizes cost while providing enough energy to complete the route is a minimum knapsack problem [38]. Let *B* represent a set of battery types, where each battery type *j* $\in B$ has a weight *w_j* in kg, energy *E_j* in kJ, and cost *C_j* in financial units. Given a decision variable *b_j* that is 1 if battery type *j* is used, and 0 otherwise, while letting $Y = E_j - \alpha \sum_{k=a}^{b-1} w_j \phi_{k(k+1)}$ and $Z = \alpha \sum_{k=a}^{b-1} y_{k(k+1)} \phi_{k(k+1)} + \beta \sum_{k=a}^{b-1} \phi_{k(k+1)}$, the minimum knapsack problem is

$$\begin{aligned}
 \min \quad & \sum_{j \in B} C_j b_j \\
 \text{s.t.} \quad & \sum_{j \in B} Y b_j \geq Z.
 \end{aligned} \tag{14}$$

The values *E*, *E_ε*, and *E/ξ* in Algorithm 2 can be replaced with $\sum_{j \in B} E_j b_j$, $\sum_{j \in B} C_j b_j$, and $\sum_{j \in B} w_j b_j$, respectively, after solving (14).

To calculate the overall delivery time, we require the delivery time and arrival time of each route, which can be found with Algorithm 3. The delivery time is when the last delivery along a route is made, while the arrival time is when the drone arrives at the depot after completing the route. Route timing

Algorithm 5 DRONECOSTANDDELIVERYTIME($\mathbf{s}, \lambda, \Phi$) Method**Input:**

$\mathbf{s}$ - The solution vector
 λ - The cost of energy for solution $\mathbf{s}$
 Φ - A boolean variable that is 1 if the goal is to minimize drone cost, or 0 to minimize overall delivery time

Output:

γ - The cost of drones
 l - The overall delivery time

```

// Get the vector  $\mathbf{u}$ , which contains pairs of delivery and
// arrival times for each route.
 $\mathbf{u} \leftarrow \text{ROUTEDELIVERYANDARRIVALTIMES}(\mathbf{s})$ 
// Perform a binary search to find the best number of
// drones  $n$  when minimizing cost.
if  $\Phi$  then
    // Set the lower and upper bounds  $\varphi$  and  $\psi$ , respectively,
    // where  $M$  is the maximum number of drones available.
     $\varphi \leftarrow 1$ 
     $\psi \leftarrow M$ 
    while  $\varphi \leq \psi - 1$  do
         $n \leftarrow \lfloor \varphi + \frac{\psi - \varphi}{2} \rfloor$ 
        if  $\text{LISTSCHEDULE}(\mathbf{u}, n) \leq T$  then
             $\psi \leftarrow n$ 
        else
             $\varphi \leftarrow n + 1$ 
     $n \leftarrow \varphi$ 
    if  $\text{LISTSCHEDULE}(\mathbf{u}, n) > T$  then
         $n \leftarrow \psi$ 
else
    // Purchase as many drones as possible to minimize time,
    // given a budget  $B$  and a cost  $F$  of a single drone.
     $n \leftarrow \lfloor \frac{B - \lambda}{F} \rfloor$ 
    // Make sure number of drones is positive.
    if  $n < 1$  then
         $n \leftarrow 1$ 
// Find the cost of drones  $\gamma$  and the overall delivery time  $l$ .
 $\gamma \leftarrow nF$ 
 $l \leftarrow \text{LISTSCHEDULE}(\mathbf{u}, n)$ 
return  $(\gamma, l)$ 

```

vector $\mathbf{u}$ contains pairs of delivery and arrival times for each route, such that element $u_i = (h_i, a_i)$, where h_i is the delivery time and a_i is the arrival time for route i . Algorithm 3 implements ROUTEDELIVERYANDARRIVALTIMES($\mathbf{s}$), which iterates over the solution $\mathbf{s}$ to calculate $\mathbf{u}$. As Algorithm 3 iterates through $\mathbf{s}$ once, it has a $\mathcal{O}(|\mathcal{N}_0|)$ bound.

To minimize the overall delivery time, we require a method of assigning R routes to n drones; this method is given in Algorithm 4. This problem is equivalent to the load balancing problem in computer science, where j jobs are assigned to m machines to minimize the maximum load of the machines. This problem is NP-hard, so we apply the list scheduling, or greedy-balance, algorithm discussed in [39, Sec. 11.1], except jobs become routes, machines become drones, and we try to minimize the maximum delivery time of the drones.

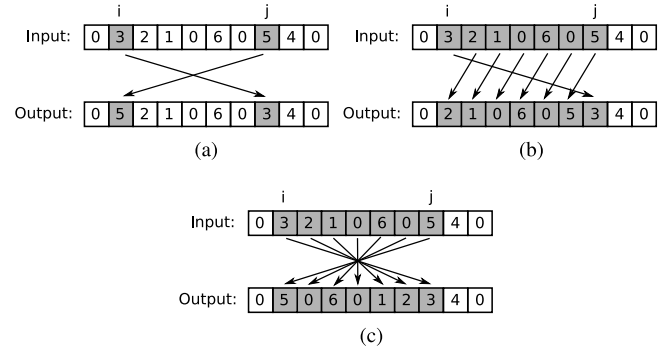


Fig. 3. Exchange rules from [37, Sec. 2.4]. (a) Swap. (b) Relocate. (c) 2-opt.

Algorithm 4 implements LISTSCHEDULE($\mathbf{u}, n$) to approximate the minimum overall delivery time. At most M drones can be purchased, and the number of elements in $\mathbf{u}$ cannot exceed the number of locations $|\mathcal{N}_0|$. Drone timing vector κ contains pairs of delivery and arrival times for each route, such that element $\kappa_i = (h_i, a_i)$. Assuming that κ is implemented as a binary heap, the bound of Algorithm 4 is $\mathcal{O}(|\mathcal{N}_0| \log M)$.

Alternatives to list scheduling, such as the longest processing time (LPT), or sorted-balance, algorithm discussed in [39] could be applied. LPT improves list scheduling by choosing the jobs to assign based on a specific order, such as by descending runtime, resulting in a better performance guarantee of 1.5 instead of 2. The order of the route delivery and arrival times in $\mathbf{u}$ depend on the order of the routes in $\mathbf{s}$. The SA algorithm randomly adjusts $\mathbf{s}$: in addition to changing the routes themselves, it also changes their order. As the SA algorithm is already adjusting the order of the routes, we choose not to implement LPT.

Algorithm 5 implements the DRONECOSTANDDELIVERYTIME($\mathbf{s}, \lambda, \Phi$) method, where λ is the cost of energy, to calculate the cost of drones γ and the overall delivery time l . When solving the MC-DDP, it performs a binary search to reduce the number of drones n as much as possible, without violating the delivery time constraint. While solving the MT-DDP, on the other hand, it sets n as high as possible without violating the budget constraint. After finding n , and in turn γ , it uses Algorithm 4 to find l . As Algorithm 5 performs a binary search to find the number of drones, calling Algorithm 4 every iteration, it has a $\mathcal{O}(|\mathcal{N}_0| \log^2 M)$ bound.

B. Simulated Annealing Algorithm

Algorithm 6 provides the SA algorithm we use. It requires an initial temperature Υ_0 , final temperature Υ' , cooling factor μ , and the number of rounds Λ per cooling phase. It is based on the approach discussed in [37, Sec. 2.5], with slight modifications. We do not include the heating phase, and instead let Υ_0 be an input to the algorithm. We use Algorithm 1 to compare the costs of different solutions.

In Algorithm 6, the exchange rules are used to find a neighboring solution $\mathbf{s}'$ from the current solution $\mathbf{s}$. The three exchange rules in [37, Sec. 2.4] are applied in Algorithm 6, each using two uniform randomly selected indices. The swap rule in Fig. 3(a) swaps two randomly selected elements in the

Algorithm 6 SIMULATEDANNEALING($\Upsilon_0, \Upsilon', \mu, \Lambda$) Method**Input:**

Υ_0 - The initial temperature
 Υ' - The final temperature
 μ - The amount that temperature is adjusted each iteration
 Λ - The number of adjustments to the solution attempted per iteration

Output:

s - A solution to the DDP

```

// Generate a random solution to the DDP using the number
// of locations  $|\mathcal{N}_0|$ . It can be infeasible at this point.
 $s \leftarrow \text{RANDOMSOLUTION}(|\mathcal{N}_0|)$ 
// Set the current temperature  $\Upsilon$  to the initial temperature
//  $\Upsilon_0$ .
 $\Upsilon \leftarrow \Upsilon_0$ 
// Perform annealing on the current solution  $s$ .
while  $\Upsilon > \Upsilon'$  do
   $\Upsilon \leftarrow \mu\Upsilon$ 
  for  $k \in \{1 \dots \Lambda\}$  do
    // Pick two random indices for  $s$ .
     $i \leftarrow \text{UNIFORMRANDOMINTEGER}(1, \text{LENGTH}(s)-1)$ 
     $j \leftarrow \text{UNIFORMRANDOMINTEGER}(1, \text{LENGTH}(s)-1)$ 
    // Choose a random exchange rule  $R$ , then adjust  $s$ .
     $R \leftarrow \text{UNIFORMRANDOMINTEGER}(1, 3)$ 
     $s' \leftarrow \text{ADJUSTWITHEXCHANGERULE}(s, R, i, j)$ 
    // Use Metropolis algorithm to determine if the
    // adjustment should become the current solution.
     $X \leftarrow \text{UNIFORMRANDOMFLOAT}(0, 1)$ 
    if  $\exp\left(-\frac{\text{COST}(s') - \text{COST}(s)}{\Upsilon}\right) \geq X$  then
       $s \leftarrow s'$ 
  return  $s$ 

```

vector, the relocate rule in Fig. 3(b) changes the position of a randomly selected element in the vector, while the 2-opt rule in Fig. 3(c) reverses a randomly chosen subvector. Note that the vectors in Fig. 3 represent the solution vector s , whose elements are the locations in the area of interest. To determine if the neighboring solution should become the current one, we apply the Metropolis algorithm [40].

Section VI compares the performance of the MILP implementations to the SA implementations to validate the cost function and demonstrate that the SA implementation can find near-optimal solutions to small DDP instances. One weakness of the SA approach is that it does not take advantage of characteristics unique to VRPs. For example, it does not take advantage of geographical information to reduce the likelihood of connecting distant locations, even though a route containing such locations would likely be infeasible. It also does not keep track of how often the swap, relocate, and 2-opt operations improve the solution, which could be used to increase the frequency of operations that are more likely to improve the solution. While heuristics that implement these approaches can likely find better solutions than SA given the same runtime, we do not believe it is likely that the trends observed in Section VI will change significantly with the heuristic. The trends can be explained by adjustments made to parameters such as area size

and the number of customer locations in an area, and do not appear to be a result of behavior specific to SA.

VI. RESULTS

In this section, we validate the multirotor helicopter energy consumption model, and analyze the performance of the MILP and SA implementations of the DDPs. We demonstrate that the SA implementations find near-optimal solutions to small instances of the DDPs, and provide consistent results for larger instances. We show that the cost found by the MC-DDP has an inverse exponential relationship with the delivery time limit, and that the overall delivery time found by the MT-DDP has an inverse exponential relationship with the budget. We demonstrate that the majority of costs go toward drones, and that changing the budget or delivery time limit causes the SA DDPs to adjust the number of drones while leaving energy costs approximately constant. We demonstrate that reusing drones to perform multiple trips is an important strategy for reducing costs. Our results indicate that optimizing battery weight is an effective strategy for reducing the total cost and overall delivery time.

To understand how the DDPs behave in general, we average the results of a number of randomly generated instances of each scenario. Each scenario has an area size of 0.25 or 1 km². Small scenarios have 6–8 delivery locations, while large scenarios have 125 or 500 delivery locations. We generate 50 random instances for each scenario. In each instance, delivery locations are uniformly distributed throughout the area and are given uniform random demands of 0.5–2 kg; depots are located in the middle of the area. We run the SA algorithm 20 times per instance, and calculate the minimum, mean, standard deviation, and average runtime of these 20 runs. The minimums, means, etc., in this section are averaged over all 50 instances of each scenario. Unless mentioned otherwise, we use the average minimum values in tables and graphs. We run the MILP implementation once per instance, and provide the average result and runtime over all 50 instances of a scenario. Unless mentioned otherwise, when running the SA algorithm, the initial temperature $\Upsilon_0 = 1$, the final temperature $\Upsilon' = 0.001$, the cooling factor $\mu = 0.99$, and the number of rounds $\Lambda = 1000$.

We assume that each drone costs $F = \$500$ and has a maximum carrying capacity of $Q = 3$ kg. Its maximum carrying capacity includes its battery and payload weight, but not its frame weight. Drones can vary in price from under \$100 for basic models to tens of thousands of dollars for industrial or military use. We assume that the operator either manufactures its own drones, or uses a hobbyist drone to make deliveries, hence the \$500 cost. A carrying capacity of 3 kg should be possible for drones designed to deliver small packages. We assign a slope $\alpha = 0.217$ kW/kg and a y-intercept $\beta = 0.185$ kW to the energy consumption model, based on results obtained from the hexacopter power consumption measurements made in Section VI-A. When delivering packages, we assume that the drone flies at a constant speed of $v = 6$ m/s between locations, and takes $\tau = 60$ s at each location to descend, deliver the package, and ascend. Batteries are given an energy density of $\xi = 650$ kJ/kg and cost $\epsilon = 0.1$ \$/kJ,

based on the energy density and price of lithium polymer batteries in U.S. dollar values at the time this paper was written.

We use the percent improvement p to compare how a value changes from x to x' after adjusting a parameter or algorithm. We express the percent improvement as

$$p = \frac{x - x'}{x'} \quad (15)$$

We either minimize the cost or the overall delivery time, so when $p > 0$, x' is an improvement over x .

The rest of this section is organized as follows. Section VI-A validates our energy consumption model by comparing it to measurements from a 3D Robotics ArduCopter Hexa-B hexacopter. Section VI-B compares the results and runtimes of the MILP and SA DDP implementations for instances with 6–8 locations, and measures the consistency and runtimes of the SA DDPs for instances of up to 500 locations. Section VI-C examines how the cost and overall delivery time found by the MC-DDP and MT-DDP, respectively, change with the delivery time limit, budget, area size, and number of locations. Section VI-C also examines how the DDPs adjust costs for different delivery time limits and budgets. Section VI-D investigates the importance of reusing drones, while Section VI-E looks into the advantages of optimizing battery weight.

A. Hexacopter Energy Consumption Measurements

In this section, we conduct two experiments to validate our energy consumption model. In the first experiment we verify that the power consumed during hover, level flight, and changing altitude is, on average, approximately equal. The second experiment verifies that the relationship between power consumption and battery and payload weight is approximately linear, and provides us with realistic values for our linear model.

Experimental results were generated with a 3D Robotics ArduCopter Hexa-B hexacopter. It has six 850 RPM/V motors with 10×4.7 APC slow fly propellers, uses an Arduino flight controller with firmware version 3.0.1 of the APM 2.5 autopilot, and has a u-blox LEA-6 GPS with a ceramic 4 mm GPS patch antenna. Unless mentioned otherwise, the hexacopter is powered by a 4-cell 14.8 V lithium polymer battery. Power consumption is calculated using current and voltage readings taken from the hexacopter's power management system, which were recorded by the ArduPilot mission planning software.

Power consumption was measured during level flight, hover, and altitude changes. The ArduPilot software under default settings was used to control the hexacopter and log the measurements. During level flight, the drone flew six times between two waypoints positioned about 94 m apart, with an average speed of about 6 m/s; the path can be seen in Fig. 4(a). During hover, the hexacopter remained as stationary as it could over a single location over a period of 40 s. During altitude changes, the hexacopter's altitude changed as in Fig. 4(b), with minimal changes in latitude and longitude.

Table I contains the average voltage, current, and power consumption during hover, level flight, and altitude adjustment.

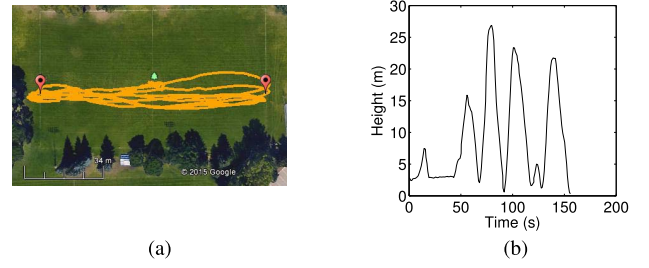


Fig. 4. Hexacopter maneuver information. (a) Hexacopter path during the straight flight maneuver. Map data and image by Google. (b) Hexacopter altitude while performing altitude changes.

TABLE I
AVERAGE VOLTAGE, CURRENT, AND POWER CONSUMED BY THE HEXACOPTER WHILE PERFORMING DIFFERENT MANEUVERS

Manoeuvre	Voltage (V)	Current (A)	Power (W)
Hover	15.6	19.9	311
Straight Flight	15.3	19.8	304
Altitude Change	14.7	20.3	297

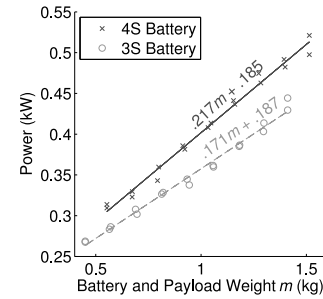


Fig. 5. Hexacopter energy consumption for various battery and payload weights m , with corresponding energy consumption model values.

It shows that, on average, the power consumed when performing the three maneuvers is approximately equal. The current is slightly lower during level flight, possibly due to translational lift, where air moving horizontally along the rotors creates extra lift. The voltages in Table I are slightly different between tests, depending on the battery charge remaining. The altitude change tests had slightly lower voltages because they were conducted last.

The second set of experiments measure the power consumption of the hexacopter as it carries different weights, while hovering 3–5 m above the ground at the same location. Measurements were repeated twice for each weight and battery type. The hexacopter was tested with both a 3-cell 11.1 V lithium polymer battery and a 4-cell 14.8 V lithium polymer battery.

Fig. 5 confirms that the power consumption of the hexacopter varies approximately linearly with battery and payload weight, for both 3- and 4-cell batteries. The power consumed by the 4-cell battery grows faster with weight than with the 3-cell battery because we used 10×4.7 slow fly propellers, which perform less efficiently at the higher motor RPM from using the 4-cell batteries.

TABLE II
PERFORMANCE OF MILP AND SA IMPLEMENTATIONS
OF THE MT-DDP

Scenario A (km ²)	$ \mathcal{N}_0 $	MILP		SA		
		Result (min)	Runtime (s)	Mean (min)	Std (min)	Runtime (s)
0.250	6	5.40	0.97	5.48	0.09	0.24
1.000	6	7.20	0.96	7.28	0.10	0.25
0.250	7	6.59	8.41	6.68	0.09	0.28
1.000	7	8.65	6.13	8.76	0.14	0.28
0.250	8	7.29	116.98	7.43	0.13	0.30
1.000	8	9.63	86.87	9.83	0.22	0.31

TABLE III
PERFORMANCE OF MILP AND SA IMPLEMENTATIONS
OF THE MC-DDP

Scenario A (km ²)	$ \mathcal{N}_0 $	MILP		SA		
		Result (k\$)	Runtime (s)	Mean (k\$)	Std (k\$)	Runtime (s)
0.250	6	0.99	9.47	1.00	0.00	0.19
1.000	6	1.04	4.84	1.04	0.00	0.19
0.250	7	1.04	75.38	1.04	0.00	0.22
1.000	7	1.10	24.84	1.11	0.00	0.22
0.250	8	1.04	1116.87	1.04	0.00	0.24
1.000	8	1.22	205.52	1.26	0.04	0.24

B. Performance of MILP and SA Implementations

This section analyzes the performance of the MILP and SA implementations of the DDPs. It compares the results and runtimes of the SA implementations with the MILP implementations, and examines how the SA implementations perform at different cooling rate levels. The MILP implementations were run in CPLEX in a single thread, otherwise with default settings. Each scenario has its own number of locations $|\mathcal{N}_0|$, not including the depot, and area size A in km².

Tables II and III show the average results and runtimes of the MILP and SA implementations of the MT-DDP and MC-DDP, respectively. The SA algorithm parameters are set to $\Upsilon_0 = 1$, $\Upsilon' = 0.001$, $\mu = 0.9$, and $\Lambda = 1000$. MT-DDP scenarios have a budget B of \$1500, while MC-DDP scenarios have delivery time limits T of 10 min.

The SA implementation consistently finds near-optimal solutions for problems with eight or less locations. SA solutions to the MT-DDP are at most seconds away from optimal, while SA solutions to the MC-DDP are at most tens of dollars away, when optimal solutions are in minutes and thousands of dollars, respectively. While optimality cannot be guaranteed when using SA, it appears to be an effective approach for solving small cases of the DDPs, providing similar results in under a second of runtime.

The runtime of the MILP implementation can grow exponentially with the number of locations, as can be observed from the runtimes in Tables II and III. In Table III, for example, the runtime increases from 75.38 to 1116.87 s when solving an eight location problem with an area of 0.25 km² instead of a seven location problem. The runtime of the SA implementation is consistently about 0.2–0.3 s for each scenario.

In Tables II and III, increasing A without changing $|\mathcal{N}_0|$ can actually cause the runtime of the MILP implementation to decrease. Locations are further apart, reducing the number

TABLE IV
SA RESULTS FOR THE MT-DDP

A (km ²)	$ \mathcal{N}_0 $	$\mu = 0.9$			$\mu = 0.99$		
		Mean (min)	Std (min)	Runtime (s)	Mean (min)	Std (min)	Runtime (s)
0.250	125	13.28	0.13	4.00	12.19	0.08	40.79
1.000	125	17.49	0.31	4.12	15.62	0.14	41.88
0.250	500	75.49	0.67	18.20	69.68	0.25	178.17
1.000	500	117.82	3.75	18.45	104.75	0.47	181.41

TABLE V
SA RESULTS FOR THE MC-DDP

A (km ²)	$ \mathcal{N}_0 $	$\mu = 0.9$			$\mu = 0.99$		
		Mean (k\$)	Std (k\$)	Runtime (s)	Mean (k\$)	Std (k\$)	Runtime (s)
0.250	125	13.99	0.23	5.82	13.52	0.21	57.91
1.000	125	17.03	0.29	6.04	16.21	0.24	58.22
0.250	500	60.40	1.02	49.31	54.55	0.43	461.27
1.000	500	75.13	1.38	57.11	65.57	0.55	497.65

of feasible routes and consequently the number of feasible solutions. Perhaps the solver improves the runtime by pruning solutions with routes that consume too much energy, carry too much weight, exceed the delivery time limit, or fail to meet some other criteria.

Tables IV and V show the results of the SA implementation at different cooling rate levels μ for scenarios with over 125 locations. As before, $\Upsilon_0 = 1$, $\Upsilon' = 0.001$, and $\Lambda = 1000$. The cooling rate level μ , however, is adjusted this time to see how much the results improve with additional runtime. MT-DDP scenarios have a budget B of \$10 000, while MC-DDP scenarios have delivery time limits T of 10 min.

The SA implementation results are consistent for large scenarios. In Table IV, the standard deviations are generally in seconds when the mean values are several minutes long. The standard deviation is always under \$1500 in Table V, and usually just a few hundred dollars, where the means are often tens of thousands of dollars. The standard deviations are greatest when $|\mathcal{N}_0|$ is high and μ is low. In this case the solution vector $\mathbf{s}$ is long, increasing the number of neighboring solutions that can be found after an adjustment, and μ is low so the temperature decreases rapidly, giving fewer opportunities to make adjustments.

One possible reason for the small standard deviation is that the majority of costs appear to come from purchasing drones, which are considerably more expensive than energy. It seems like the SA DDPs consistently find similar numbers of drones, and differ more with regards to energy consumption. The variation in energy consumption is dampened by the relatively low energy costs, leading to the low standard deviations.

Increasing the cooling rate level from 0.9 to 0.99 resulted in percent improvements of up to 15% for the means in the scenarios tested, which came at the cost of increasing runtime by about a factor of 10. Even though increasing the cooling rate level generally improves results, the improvement is not significant compared to the increase in runtime. While not shown in this paper, increasing μ from 0.99 to 0.999 resulted in percent improvements of up to 5%, again with the runtime increasing by about a factor of 10. We therefore set $\mu = 0.99$ for the rest of this section, as further increasing μ would result in minor improvements compared to the increase in runtime.

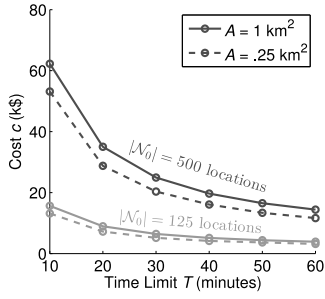


Fig. 6. Costs found by the SA MC-DDP algorithm for various delivery time limits, area sizes, and numbers of nodes.

We would like to mention that heuristics or exact algorithms that take advantage of characteristics unique to VRPs may find lower costs and times than the SA approach in this paper. An approach that takes advantage of geographical information to reduce the likelihood of connecting far apart locations would likely find better results as fewer infeasible routes would be tested. The number of times that performing operations such as swap, relocate, or 2-opt improve the solution could be tracked; operations which tend to improve the solution could be chosen more frequently than operations that do not.

While such approaches may lower cost, we do not believe that the trends we observed will change significantly with a different method. The trends can be explained by adjustments made to parameters such as area size and the number of customer locations in an area. They do not appear to be a result of behavior specific to SA.

C. Analysis of DDP Parameters

This section examines how adjusting the budget B in dollars, time limit T in minutes, number of locations $|\mathcal{N}_0|$, and area size A in km^2 affects the results obtained by the SA implementations of the DDPs. We show that the overall delivery time l found by the MT-DDP is inversely proportional to B , and that the cost c found by the MC-DDP is inversely proportional to T . We also examine the sensitivity of the DDPs to changes in B , T , $|\mathcal{N}_0|$, and A . This information may be used to find the point where guaranteeing a better delivery time limit is not worth the additional cost, or alternatively the point at which reducing the budget is not worth the increase in overall delivery time.

Fig. 6 shows how adjustments to T , $|\mathcal{N}_0|$, and A affect c , while Fig. 7 examines how adjustments to B , $|\mathcal{N}_0|$, and A affect l . To help explain the changes to l and c , Figs. 8(a) and (b) provide stacked bar graphs depicting the split in c between drones and energy for scenarios with 500 locations in a 1 km^2 area.

Fig. 6 shows an inverse exponential relationship between c and T . Changes to c from adjusting T are larger when T is low; from Fig. 8(a) we can see that this change comes mostly from adding or removing drones. When T is low, most drones fly short routes and are seldom reused; as T increases, drone reuse and route length increase, while the number of routes decreases. This combination of factors reduces the

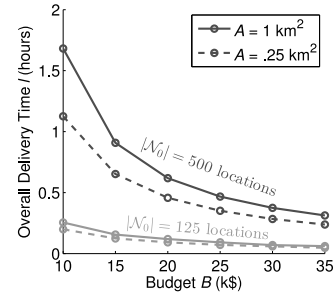


Fig. 7. Overall delivery times found by the SA MT-DDP algorithm for various budgets, area sizes, and numbers of nodes.

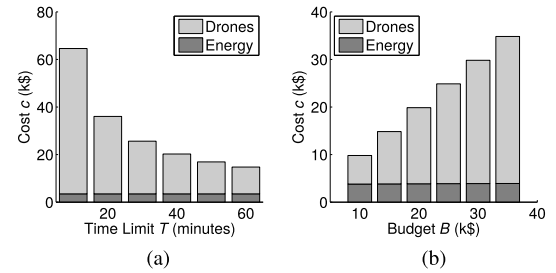


Fig. 8. Costs found by the SA algorithms for various delivery time limits and budgets, with 500 locations in a 1 km^2 area. (a) MC-DDP cost breakdown. (b) MT-DDP cost breakdown.

number of purchased drones. After a certain point, increasing T does not alter the routes as much, meaning costs are mainly reduced by reusing drones. The greater reductions in cost for low delivery times likely come from not only being able to reuse drones, but also from increasing the length and consequently reducing the number of routes flown.

Fig. 7 shows an inverse exponential relationship between l and B . Changes to l from adjusting B are greater when B is low. From Fig. 8(b), we can see that, as with the MC-DDP, this change comes mostly from adding or removing drones. When B is low, only a small number of drones can be purchased to complete deliveries. Adding an additional drone reduces the delivery time considerably; for example, suppose a single drone is making all of the deliveries, then a second drone is added. If their burden is shared as evenly as possible, l can potentially be halved. Now suppose a large number of drones are available, and another one is added. The routes of each drone could perhaps be shortened slightly to give the additional drone a new route, but this will likely not have anywhere near the impact on l compared to adding a second drone when only a single drone is making deliveries.

Figs. 6 and 7 show that when $|\mathcal{N}_0|$ is high, the MC-DDP becomes more sensitive to changes in T , and the MT-DDP becomes more sensitive to changes in B . For a given increase in T with the MC-DDP, more routes can be extended and additional drones can be reused when $|\mathcal{N}_0|$ is high. Similarly, increasing $|\mathcal{N}_0|$ when T is low has a greater effect on cost than increasing $|\mathcal{N}_0|$ when T is high. When T is low, routes are short and drones are reused less, so increasing $|\mathcal{N}_0|$ requires purchasing more drones than if T were high. For higher values

TABLE VI
MC-DDP COSTS WHEN REUSE IS ENABLED AND DISABLED FOR
VARIOUS TIME LIMITS IN A 0.25 km² AREA WITH 500 NODES

Delivery Time Limit (minutes)	Cost (k\$)		Percent Improvement (%)
	Reuse	No Reuse	
10.0	54.6	112.9	106.84
20.0	29.2	112.7	286.38
30.0	20.6	112.8	446.65
40.0	16.2	112.8	597.77
50.0	13.6	112.7	730.95
60.0	11.7	112.7	866.62

of $|\mathcal{N}_0|$ with the MT-DDP, drones require more time to complete deliveries, as they must travel further and spend more time servicing locations. When B is low, only a small number of drones are available and must therefore be reused more frequently, further increasing l .

From Figs. 6 and 7, we see that the DDPs are less sensitive to changes in A than to changes in $|\mathcal{N}_0|$. Increasing A increases the distance traveled between locations, and consequently the time and energy required to complete deliveries. Changing $|\mathcal{N}_0|$ not only increases the travel distance, but also the time spent servicing locations, as well as the number of deliveries that must be made. Changing $|\mathcal{N}_0|$ therefore has a greater effect on time and cost than A .

Note that these results may not hold for much higher values of A or $|\mathcal{N}_0|$. If A or $|\mathcal{N}_0|$ are so high that it becomes impossible in some scenarios to avoid violating the capacity constraint, cost constraint, or timing constraint, then the average price given by the SA algorithm could greatly increase. If a constraint is violated, the SA algorithm makes that solution undesirable by increasing its cost.

D. Effect of Drone Reuse

In the previous section, we learned that the majority of costs come from purchasing drones. Unlike other VRPs that model fuel consumption, the DDPs permit drones to be reused to perform multiple trips. To demonstrate the importance of allowing drone reuse, we show that our SA implementation performs better when drone reuse is permitted.

To look at the importance of drone reuse, we altered the SA implementation in Section V-A by removing the binary search in Algorithm 5, and setting the number of drones n to the number of routes. Table VI compares the results of the MC-DDP with and without drone reuse. We did not include a similar table for the MT-DDP, as solutions could not be found for budgets of \$35 000 or less.

Table VI shows substantial gains when allowing drone reuse. We can see that without reusing drones a limit of about \$113 000 is reached. By reusing drones, we can pass this limit without exceeding the delivery time constraints.

Note that we are ignoring the cost of maintaining drones. Flying fewer drones will likely increase maintenance costs due to increasing the total time flown per drone. New drones are generally expensive, and worn components can presumably be replaced quickly and relatively inexpensively, so reducing the number of drones while increasing the costs of labor and replacement parts is likely the less expensive option.

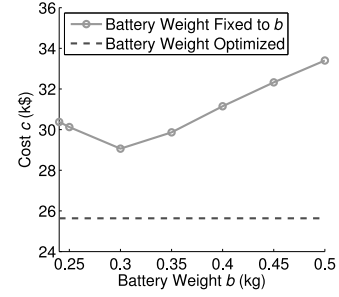


Fig. 9. Costs found by the MC-DDP when every drone's battery weighs b kg, with a time limit of 30 min on a 1 km² field with 500 nodes.

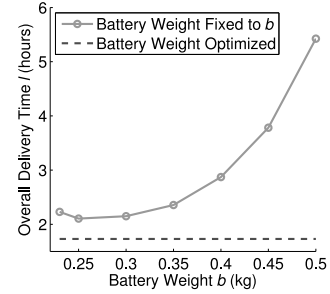


Fig. 10. Delivery times found by the MT-DDP when every drone's battery weighs b kg, with a budget of \$10 000 on a 1 km² field with 500 nodes.

E. Effect of Optimizing Battery Weight

While papers on minimizing fuel consumption do not reuse drones, MTRVPs do. We are not aware of any MTRVPs that try to optimize battery weight and therefore the energy given to each drone. Instead, they may limit each drone's travel distance or travel time to a predefined value. In this section, we apply our energy model to an MTRVP, and fix the battery weight for every route to the same constant value b in kg. Similar to limiting distance or time, the amount of energy that can be consumed is limited. We show that optimizing battery weight for each route may provide better costs and delivery times compared to optimizing a fixed battery weight that is identical for each route. We also demonstrate that even if all routes share an identical battery weight, optimizing it can still be important.

To look at the effect of optimizing battery weight, we altered Algorithm 2 to assume that every path has an identical battery weight of b kg. The energy in a battery is then ξb , where ξ is the energy density of the battery in kJ/kg. Fig. 9 shows the cost c found by the MC-DDP when assuming an identical battery weight b for each route. The dashed line represents the cost found by optimizing battery weights for each route with the unaltered version of Algorithm 2; it is flat because it does not consider b . Fig. 10 is similar, but for the overall delivery time l found by the MT-DDP.

Figs. 9 and 10 demonstrate that optimizing battery weights for each route to give drones exactly enough energy to complete their upcoming routes can provide substantial savings compared to optimizing a fixed battery weight that is identical for every route. In Fig. 9, the percent improvement when optimizing each route's battery weight amounts to 13% compared to assuming an identical battery weight of 0.3 kg for

every route. In Fig. 10, the percent improvement of optimizing each route's battery weight amounts to about 22% compared to assuming each route has a battery weight of 0.25 kg when using the MT-DDP. Note that while optimizing battery weight for each route provides savings compared to optimizing a fixed battery weight, it can be less effective with a small area size or low number of nodes.

Even if the battery weight is identical for all routes, optimizing it can improve cost and delivery time, as can be seen in Figs. 9 and 10. Setting b to 0.3 kg instead of 0.5 kg provides a percent improvement of 15% for the MC-DDP, and setting b to 0.25 kg instead of 0.45 kg provides a percent improvement of 80% for the MT-DDP.

VII. CONCLUSION

In this paper, we derived an energy consumption model for multirotor drones, provided a linear approximation for it, and verified the approximation experimentally. With this approximation we proposed the DDPs that minimize cost or delivery time while considering battery weight, payload weight, and drone reuse, and implemented them as MILPs. To solve practical scenarios with hundreds of locations, we proposed a string-based SA algorithm for solving the DDPs.

Numerical results indicate that optimizing battery weight and reusing drones are important considerations for drone delivery. Optimizing battery weights resulted in percent improvements of over 10% compared to solutions where each drone had an identical battery weight. Even when drones had an identical battery weight, optimizing that weight provided a percent improvement of 80% for the MT-DDP in the scenario considered in Section VI-E. In Section VI-D, reusing drones led to costs in the tens of thousands of dollars, while preventing drone reuse resulted in costs of approximately \$113 000.

When minimizing costs with the MC-DDP, an inverse exponential relationship between the cost and the delivery time limit was noticed. An inverse exponential relationship was also seen between the overall delivery time and the budget constraint with the MT-DDP. For both DDPs, the majority of costs are allocated to drones instead of energy. This information is useful for a practitioner. For example, reducing already low delivery times can require a relatively large investment in additional drones. Adding a single drone when the number of drones is low can greatly reduce delivery times, while adding a drone when the number of drones is high has a limited effect in comparison.

One weakness of the SA algorithm is that it does not take advantage of characteristics inherent to a VRP. For instance, it does not take advantage of geographical information to reduce the likelihood of trying infeasible routes with two locations at opposite ends of the area of interest. The cost function of the SA algorithm accounts for reusing drones, as well as the effect of battery and payload weight on energy consumption, so it could be applied to a heuristic better suited to VRPs. The cost function itself is also an approximation, as it uses list scheduling to assign drones to routes. Perhaps a multilayer

approach that optimizes routes and their assignment to drones would perform better.

Additional strategies could be implemented to further reduce costs and generalize the problem. Time windows could be added to locations to ensure that packages are delivered within a specific time. Multiple depots or recharging stations could be studied as a method of extending the flight distance of drones. While we treat cost and delivery time as separate optimization objectives, a multiobjective optimization problem could combine both. Maintenance costs could be included when determining drone reuse. The performance of the MILP and SA DDPs on a cloud service such as Amazon Web Services [41] could be investigated. The runtime or resources provided to the algorithms could be adjusted to compare performance gains from increasing the service cost.

The DDPs have practical applications in delivery and emergency response scenarios. They can plan routes and assign drones to them in scenarios with tight budgets or time limits. In a disaster response scenario, for example, the MT-DDP could make the best use of a limited budget to optimize the routes taken by a fleet of drones to deliver food or medicine to assigned locations as quickly as possible. Alternatively, a delivery company could determine the budget necessary to guarantee a minimum delivery time with the MC-DDP. The SA implementations can be used to quickly predict delivery times and plan budgets for drone-based delivery operations with a large number of locations.

REFERENCES

- [1] Amazon.com Inc. *Amazon Prime Air*. Accessed on Apr. 2, 2016. [Online]. Available: www.amazon.com/primeair
- [2] DHL International GmbH. (Sep. 2014). *DHL Parcelcopter Launches Initial Operations for Research Purposes*. [Online]. Available: http://www.dhl.com/en/press/releases/releases_2014/group/dhl_parcelcopter_launches_initial_operations_for_research_purposes.html
- [3] J. Stewart. (Aug. 2014). *Google Tests Drone Deliveries in Project Wing Trials*, BBC, London, U.K. [Online]. Available: <http://www.bbc.com/news/technology-28964260>
- [4] Aljazeera. (Feb. 2014). *UAE to Use Drones for Citizen Services*. [Online]. Available: <http://www.aljazeera.com/news/middleeast/2014/02/uae-use-drones-government-services-20142121717319272.html>
- [5] C. Dillow. (May 2015). *Meet Matternet, the Drone Delivery Startup That's Actually Delivering*, Fortune, New York, NY, USA. [Online]. Available: <http://fortune.com/2015/05/01/matternet-drone-delivery/>
- [6] P. Morgan, *Carbon Fibers and Their Composites* (Materials Engineering). Boca Raton, FL, USA: CRC Press, 2005.
- [7] C. Reiter. (Nov. 2013). *BMW Makes Lone Shift to Carbon Fiber to Gain Auto Edge*, Bloomberg, New York, NY, USA. [Online]. Available: <http://www.bloomberg.com/news/articles/2013-11-14/bmw-makes-lone-shift-to-carbon-fiber-to-gain-auto-edge>
- [8] T. Reddy, *Linden's Handbook of Batteries*, 4th ed. New York, NY, USA: McGraw-Hill, 2010.
- [9] GPS.gov: Augmentation Systems. (Dec. 2015). *National Coordination Office for Space-Based Positioning, Navigation, and Timing*. [Online]. Available: <http://www.gps.gov/systems/augmentations/>
- [10] S. Karpenko, I. Konvalenko, A. Miller, B. Miller, and D. Nikolaev, "UAV control on the basis of 3D landmark bearing-only observations," *Sensors*, vol. 15, no. 12, pp. 29802–29820, Nov. 2015.
- [11] T. Merz and F. Kendoul, "Dependable low-altitude obstacle avoidance for robotic helicopters operating in rural areas," *J. Field Robot.*, vol. 30, no. 3, pp. 439–471, Apr. 2013.
- [12] R. Carloni *et al.*, "Robot vision: Obstacle-avoidance techniques for unmanned aerial vehicles," *IEEE Robot. Autom. Mag.*, vol. 20, no. 4, pp. 22–31, Dec. 2013.
- [13] I. Bekmezci, O. K. Sahingoz, and Ş. Temel, "Flying ad-hoc networks (FANETs): A survey," *Ad Hoc Netw.*, vol. 11, no. 3, pp. 1254–1270, May 2013.

- [14] J. Li, Y. Zhou, and L. Lamont, "Communication architectures and protocols for networking unmanned aerial vehicles," in *Proc. IEEE GC Wkshps*, Atlanta, GA, USA, Dec. 2013, pp. 1415–1420.
- [15] C. C. Murray and A. G. Chu, "The flying sidekick traveling salesman problem: Optimization of drone-assisted parcel delivery," *Transp. Res. C Emerging Technol.*, vol. 54, pp. 86–109, May 2015.
- [16] S. Erdoğan and E. Miller-Hooks, "A green vehicle routing problem," *Transp. Res. E Logistics Transp. Rev.*, vol. 48, no. 1, pp. 100–114, Jan. 2012.
- [17] É. D. Taillard, G. Laporte, and M. Gendreau, "Vehicle routing with multiple use of vehicles," *J. Oper. Res. Soc.*, vol. 47, no. 8, pp. 1065–1070, Aug. 1995.
- [18] Y. Xiao, Q. Zhao, I. Kaku, and Y. Xu, "Development of a fuel consumption optimization model for the capacitated vehicle routing problem," *Comput. Oper. Res.*, vol. 39, no. 7, pp. 1419–1431, Jul. 2012.
- [19] International Business Machines Corporation. *CPLEX Optimizer*. Accessed on Apr. 2, 2016. [Online]. Available: <http://www.ibm.com/software/commerce/optimization/cplex-optimizer/>
- [20] K. Sundar and S. Rathinam, "Algorithms for routing an unmanned aerial vehicle in the presence of refueling depots," *IEEE Trans. Autom. Sci. Eng.*, vol. 11, no. 1, pp. 287–294, Jan. 2014.
- [21] F. Guerriero, R. Surace, V. Loscrì, and E. Natalizio, "A multi-objective approach for unmanned aerial vehicle routing problem with soft time windows constraints," *Appl. Math. Model.*, vol. 38, no. 3, pp. 839–852, Feb. 2014.
- [22] J.-F. Cordeau, G. Laporte, M. W. Savelsbergh, and D. Vigo, "Chapter 6 vehicle routing," in *Transportation (Handbooks in Operations Research and Management Science)*, vol. 14, C. Barnhart and G. Laporte, Eds. Amsterdam, The Netherlands: Elsevier, 2007, pp. 367–428.
- [23] B. Eksioğlu, A. V. Vural, and A. Reisman, "The vehicle routing problem: A taxonomic review," *Comput. Ind. Eng.*, vol. 57, no. 4, pp. 1472–1483, Nov. 2009.
- [24] M. Schneider, A. Stenger, and D. Goeke, "The electric vehicle-routing problem with time windows and recharging stations," *Transp. Sci.*, vol. 48, no. 4, pp. 500–520, Nov. 2014.
- [25] G. Hiermann, J. Puchinger, S. Ropke, and R. F. Hartl, "The electric fleet size and mix vehicle routing problem with time windows and recharging stations," *Eur. J. Oper. Res.*, vol. 252, no. 3, pp. 995–1018, Aug. 2016.
- [26] B. Fleischmann, "The vehicle routing problem with multiple use of the vehicles," Working paper, Fachbereich Wirtschaftswissenschaften, Universität Hamburg, Hamburg, Germany, 1990.
- [27] N. Azi, M. Gendreau, and J.-Y. Potvin, "An adaptive large neighborhood search for a vehicle routing problem with multiple routes," *Comput. Oper. Res.*, vol. 41, pp. 167–173, Jan. 2014.
- [28] D. Cattaruzza, N. Absi, D. Feillet, and T. Vidal, "A memetic algorithm for the multi trip vehicle routing problem," *Eur. J. Oper. Res.*, vol. 236, no. 3, pp. 833–848, Aug. 2014.
- [29] M. Cheikh, M. Ratli, O. Mkaouer, and B. Jarboui, "A variable neighborhood search algorithm for the vehicle routing problem with multiple trips," *Electron. Notes Discrete Math.*, vol. 47, pp. 277–284, Feb. 2015.
- [30] F. Hernandez, D. Feillet, R. Giroudeau, and O. Naud, "Branch-and-price algorithms for the solution of the multi-trip vehicle routing problem with time windows," *Eur. J. Oper. Res.*, vol. 249, no. 2, pp. 551–559, Mar. 2016.
- [31] İ. Kara, B. Y. Kara, and M. K. Yetis, "Energy minimizing vehicle routing problem," in *Combinatorial Optimization and Applications (LNCS 4616)*, A. Dress, Y. Xu, and B. Zhu, Eds. Berlin, Germany: Springer, 2007, pp. 62–71.
- [32] Japanese Ministry of Land, Infrastructure, Transport and Tourism. (2009). *Fuel Consumption and CO2 Emission of Passenger Cars*. [Online]. Available: <http://www.mlit.go.jp/common/000037099.pdf>
- [33] E. E. Zachariadis, C. D. Tarantilis, and C. T. Kiranoudis, "The load-dependent vehicle routing problem and its pick-up and delivery extension," *Transp. Res. B Meth.*, vol. 71, pp. 158–181, Jan. 2015.
- [34] J. G. Leishman, *Principles of Helicopter Aerodynamics* (Cambridge Aerospace Series), 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 2002.
- [35] J. R. Montoya-Torres, J. L. Franco, S. N. Isaza, H. F. Jiménez, and N. Herazo-Padilla, "A literature review on the vehicle routing problem with multiple depots," *Comput. Ind. Eng.*, vol. 79, pp. 115–129, Jan. 2015.
- [36] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, May 1983.
- [37] I. Kaku, Y. Xiao, and G. Xia, "The deterministic annealing algorithms for vehicle routing problems," *Int. J. Smart Eng. Syst. Design*, vol. 5, no. 4, pp. 327–339, 2003.
- [38] H. Kellerer, U. Pferschy, and D. Pisinger, *Knapsack Problems*. Berlin, Germany: Springer, 2004.
- [39] J. Kleinberg and É. Tardos, *Algorithm Design*. Boston, MA, USA: Addison-Wesley, 2011.
- [40] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, and E. Teller, "Equation of state calculations by fast computing machines," *J. Chem. Phys.*, vol. 21, no. 6, pp. 1087–1092, Jun. 1953.
- [41] Amazon Web Services Inc. (2016). *Amazon Web Services*. [Online]. Available: <https://aws.amazon.com/>



include wireless sensor systems.



Group, Calgary, responsible for radio channel propagation measurements and simulating the physical layer performance of high-speed CDMA and multiple antenna wireless systems. He is currently a Professor with the Department of Electrical and Computer Engineering, University of Calgary. His research interests include data networks, physical layer communications, and communications channel propagation measurements.



As part of his industrial experience in Nortel Networks, Mississauga, ON, Canada, PMC-Sierra, Sunnyvale, CA, USA, and Protolinx Corporation, Toronto, ON, Canada. He has researched on CMOS device modeling, high-speed mixed-signal IC design, and data networks. His current research interests include analog/digital CMOS circuit design, communication systems, and biomedical instrumentation.

Kevin Dorling (S'14) received the B.S. degree in computer engineering (with distinction) from the University of Calgary, Calgary, AB, Canada, in 2010, where he is currently pursuing the Ph.D. degree in electrical engineering.

He was an Intern with Bell Labs, Alcatel-Lucent, Stuttgart, Germany, in 2011 and 2013, focusing on deployment techniques for wireless sensor networks. In 2009, he was an Intern with CDL Systems, Calgary, developing control station software for unmanned vehicles. His current research interests

Jordan Heinrichs received the B.S. degree in electrical engineering (with distinction) from the University of Calgary, Calgary, AB, Canada, in 2016, with a focus on computer engineering.

He tested multicopters as a summer research student in 2013. From 2014 to 2015, he was an Intern with Lockheed Martin CDL Systems, Calgary, AB, Canada, developing software for miniature unmanned aerial vehicles. He is currently with Hitachi ID Systems, Calgary, developing software for identity management.

Geoffrey G. Messier (S'91–M'98) received the B.S. degrees in electrical engineering and computer science (with great distinction) from the University of Saskatchewan, Saskatoon, SK, Canada in 1996, the M.Sc. degree in electrical engineering from the University of Calgary, Calgary, AB, Canada, in 1998, and the Ph.D. degree in electrical and computer engineering from the University of Alberta, Edmonton, AB, Canada, in 2004.

From 1998 to 2004, he was with Nortel Networks

CDMA Base Station Hardware Systems Design Group, Calgary, responsible for radio channel propagation measurements and simulating the physical layer performance of high-speed CDMA and multiple antenna wireless systems. He is currently a Professor with the Department of Electrical and Computer Engineering, University of Calgary. His research interests include data networks, physical layer communications, and communications channel propagation measurements.

Sebastian Magierowski (S'93–M'04) received the Ph.D. degree in electrical engineering from the University of Toronto, Toronto, ON, Canada, in 2004.

From 2004 to 2012, he served as an Assistant/Associate Professor with the Department of Electrical and Computer Engineering, University of Calgary, Calgary, AB, Canada, after which he joined the faculty of the Department of Electrical Engineering and Computer Science, Lassonde School of Engineering, York University, Toronto.